

safeL4: Rethinking Safe Language-Based Single Address Space Microkernels

Anonymous Author(s)

Abstract

Microkernels provide good features such as high security, safety, and flexibility, but their reliance on hardware-based isolation leads to heavy performance overhead, which limits their widespread adoption. Although constructing single-address-space microkernels based on safe language-based isolation is a promising approach to addressing the performance issue, this approach still faces challenges in maintaining the principle of minimality and enforcing capability-based, fine-grained access control over system resources. We propose safeL4, an approach to constructing microkernels based on safe languages. It guarantees minimality through low-level abstractions for unsafe operations, and implements a capability system based on the safe language’s object capabilities to achieve fine-grained access control over all operations on system resources. We implement a prototype system in Rust, and our evaluation results show that the primitives provided by safeL4 are, on average, 1.9× faster than those of seL4.

CCS Concepts: • Software and its engineering → Operating systems.

Keywords: Microkernel, Safe Language, Capability System

1 Introduction

Despite performance concerns, microkernels have long been regarded as offering significant advantages over monolithic kernels [37, 56], primarily in security, safety, and flexibility. First, microkernels adhere to the principle of minimality [55], ensuring that the *trusted computing base* (TCB) is as small as possible. Second, microkernels utilize capability systems [8, 47, 76] to implement fine-grained access control over system resources, thereby enforcing the *principle of least privilege* (PoLP) [42, 70]. The minimality principle and capability-based access control enable formal verification of the kernel [23], leading to the world’s first formally verified *operating system* (OS) kernel, seL4 [45, 65, 75]. Finally, by partitioning OS services into domains that can be updated or replaced independently, microkernels achieve high flexibility. In the remainder of this paper, we refer to the domains protected by the microkernel as *protection domains* [48].

Microkernels, due to their advantages, were once envisioned as the basis for all types of operating systems [56]. However, since the first generation of microkernels [1], they have been used mainly in embedded and safety-critical environments (i.e., essentially static systems). In contrast, monolithic kernels remain the most prevalent and widely adopted

kernel architecture. Emerging scenarios, such as smart vehicles, smartphones, trusted execution environments, containers, and confidential virtual machines, have strict security and safety requirements and therefore favor microkernels [17, 60, 61, 77, 87]. These scenarios are no longer limited to embedded systems and involve running general-purpose applications that demand a broad range of OS functionality.

Nevertheless, the hardware-based isolation used by microkernels imposes significant performance overhead in these scenarios, including context switches caused by *inter-process communication* (IPC), cross-address-space data transfers [21], and increased IPC frequency [17]. Although various measures have been proposed to mitigate the performance issues of microkernels, none has fundamentally solved the problem. For example, some OS designs [14, 17] place the most critical servers and drivers in the kernel, which conflicts with the principle of minimality; moreover, some measures duplicate kernel state across multiple servers [17], introducing extra overhead for state synchronization.

In fact, there is another technical approach that aims to design microkernels that do not rely on hardware-based isolation to eliminate the performance overhead, namely, the use of type-safe languages. These languages use type systems to define clear contracts for components (such as modules, structures, and functions), ensuring that each component interacts with others only through well-specified interfaces and preventing unauthorized access [15, 54]. Safe language-based microkernels and applications run in the same address space and the same CPU mode, i.e., a *single-address-space OS* (SASOS), but they still follow the common concept of a kernel, namely, “the part of the operating system that is mandatory and common to all other software” [55]. Such a language-based approach significantly reduces the performance overhead introduced by hardware isolation: cross-domain communication is simplified to function calls, and cross-domain data transfer is achieved via zero-copy object ownership transfer. Typical representatives of safe language-based microkernels include SPIN [6, 7], Singularity [25, 40, 41], and RedLeaf [66]. There are also some kernels that, although not explicitly claiming to be microkernels, have adopted certain microkernel features in their design, such as minimizing the TCB (Asterinas [68]) and fault isolation (Tock [52, 71], Theseus [10]).

However, current safe language-based kernels face two main issues in meeting microkernel requirements. First, these kernels lack a capability system for fine-grained access control over all system resources. A capability system is the

principal mechanism microkernels use to restrict the system resources available to a protection domain (e.g., to eliminate ambient authority [64] and the confused deputy problem [33]), thereby enforcing PoLP and serving as a prerequisite for formal verification to achieve high trustworthiness. Consequently, modern microkernels such as seL4 [36], L4Re [44], and HongMeng [17] all adopt capability systems as a core mechanism. Unfortunately, conventional microkernel capability systems rely on hardware-based isolation to guarantee capability unforgeability, which incurs significant performance overhead when accessing capability-protected resources. Accessing a capability-protected kernel object in seL4 consumes 379 CPU cycles (Table 1), and this overhead primarily consists of CPU mode switching, capability lookup within the capability container (CNode), capability decoding (necessitated by the semantic gap between the user-space application and the kernel regarding capability metadata), and authority check.

Table 1. Profile of the access of a kernel object in seL4 on the Raspberry Pi 3B+ board (in CPU cycles).

Mode switch	Cap lookup	Cap decode & authority check	Others [†]	Total
139	58	66	116	379

[†] Remaining overhead (e.g., IPC-buffer lookup, extra capability lookup, and return-path scheduling).

Table 2. Comparison of safe language-based kernels in terms of minimality principle adherence and TCB size.

OS	Minimality	TCB Size (LCS)	
		core + alloc	Others [†]
SPIN [7]	✗ (memory mgr, thread mgr)	-	-
Singularity [41]	✗ (memory mgr, process mgr, IO mgr, scheduler)	-	-
Tock [52, 71]	✗ (process mgr, scheduler)	1 911	2 903
RedLeaf [66]	✗ (memory mgr, scheduler)	4 357	17 240
Theseus [10]	✗ (memory mgr, scheduler)	8 720	40 459
Asterinas [68]	✗ (phy mem mgr, scheduler)	9 442	11 090

[†] Linked TCB code other than Rust core and alloc, including kernel/framework code and its direct or indirect unsafe dependencies required by the whole OS.

The second issue is that the TCB of such kernels tends to become large, driven by two main factors. First, beyond the kernel code, the TCB includes toolchain libraries and third-party dependencies linked into the kernel image. Second, these kernels typically require all unsafe code to be encapsulated within the kernel, as allowing unsafe operations in protection domains would compromise the safe language-based isolation guarantees. To quantify the TCB size, we use the *linked code size* (LCS) metric [68], which counts the total *lines of code* (LoC) linked into the kernel image. As detailed in Table 2, our evaluation of the functionality and LCS of several popular Rust-based kernels reveals that they do not follow the minimality principle, resulting in relatively large TCBs. Although Tock exhibits a smaller LCS, this is because it is a lightweight OS for low-power microcontrollers.

To address the above issues, we propose safeL4, a Rust-based single-address-space microkernel. The core idea of

safeL4 is to construct capabilities that do not rely on hardware isolation, treating them as a first-class abstraction to uniformly provide protection domains with system resource abstractions that satisfy the microkernel minimality principle and fine-grained access control requirements. First, based on Rust’s object capabilities (i.e., object references or pointers [12, 20, 32]), safeL4 constructs a capability system that does not require address space isolation for protection. It enforces fine-grained access control over system resources within a *single address space* (SAS), thereby reducing the performance overhead caused by crossing address spaces in traditional capability systems. Second, it adopts a microkernel approach to provide low-level abstractions for hardware resources and the communication functionality, and safely encapsulates only the essential unsafe operations, thereby reducing the TCB. Third, by utilizing a *capability derivation tree* and Rust’s visibility control, it solves the problem that Rust’s native object capabilities can neither record capability derivation relationships nor be revoked. This equips the capability system with interfaces of sufficiently rich semantics to support the kernel’s management of system resources.

safeL4 demonstrates the advantages of building a microkernel using a safe language. First, by reducing the performance overhead of *inter-domain communication* (IDC), safeL4 can remove the only policy-related component in the L4 family of microkernels, the scheduler [23], from the microkernel, thus more closely adhering to the minimality principle. Second, the object capabilities provided by safe languages allow capabilities to reside directly within protection domains, eliminating the need for specialized kernel objects to contain them. This approach not only reduces the microkernel’s TCB but also simplifies capability management, which is a key source of complexity in designing certain OS services, as it involves coordination among the caller, callee, and microkernel. The contributions of this paper are summarized as follows.

- We propose an approach to constructing a capability system for system resources based on Rust’s features, which avoids the heavy overhead of hardware isolation-based capability systems and satisfies microkernels’ minimality principle as well as the requirements for capability-based fine-grained access control.
- We address the issue that Rust’s native object capabilities fail to meet the management requirements of microkernels for system resources.
- We build a prototype of safeL4 in Rust and implement core OS services. We port the musl libc, enabling applications written in unsafe languages to run in user space. Evaluation results show that safeL4’s safe abstraction achieves 1.91× higher performance than seL4 on average, with IDC performance improving by 7.8× to 22.3×.

2 Background

2.1 seL4 Capability System

The seL4 capability system governs access control for all kernel objects. Any call on a kernel object must be made through a capability for that object, which encapsulates a reference to the object and a specific set of access rights that dictate the operations an application is authorized to perform. By controlling the dissemination of these capabilities, the system enforces access control to kernel resources. The seL4 kernel uses a variant of the take-grant model [13, 22] to manage the propagation of authority via capabilities.

In seL4, capabilities are stored in the kernel to prevent tampering by user-level programs. The collection of capabilities owned by an application constitutes its capability address space (Cspace). Within this space, each capability is assigned a unique address (index) that the application uses to reference it.

2.2 Object-Capability System

An object-capability system [63, 79, 80] is a security model that combines the principles of object-oriented programming with capability-based access control [53] to manage and restrict access to resources within a software environment. In this model, objects are treated as resources, object methods serve as interfaces to access those resources, and object references act as capabilities. In Rust, for instance, references and smart pointers can fulfill this role. Several memory-safe languages and their subsets support object-capability models, including Java [59, 88], E [62], Encore, Pony, and Rust [38].

The unforgeability of capabilities is fundamental to preventing unauthorized access. Consequently, an object-capability system must guarantee that object references cannot be forged. This requirement explains why, when using Rust, only its safe subset can enforce an object-capability model: unsafe Rust allows raw pointer manipulation that could trivially bypass reference unforgeability. For this reason, safeL4 mandates that all code executing within its isolated domains be written exclusively in safe Rust.

3 Related Work and Motivation

3.1 Safe Language-Based Kernels

Constructing kernels in safe languages to reduce the overhead of hardware-based isolation has a long research history that can be divided into two phases. The first phase, spanning the 1990s and 2000s, explored using garbage-collected safe languages to build OS protection domains [5–7, 25, 29, 40, 41, 86]. Representative microkernels in this phase include SPIN [6, 7], based on the Modula-3 language [16], and Singularity [25, 40, 41], based on Sing#. Beyond providing protection domains, SPIN and Singularity also use the object capabilities of safe languages to design unforgeable tokens

for system resources, serving as proofs of authority for resource access. In addition, Singularity provides fault isolation for protection domains by enforcing that any memory shared between domains can only belong to one domain at a time (single ownership), preventing a crashing domain from affecting the rest of the system.

Contemporaneous with Singularity, there were also several OSes based on the Java language. J-Kernel [86] leverages Java’s type safety to establish boundaries between domains and designs a revocable capability for objects shared across domains, ensuring that any access from other domains to a crashed domain’s shared objects triggers an exception. KaffeOS [5] implements a process abstraction within a *Java virtual machine* (JVM) and provides isolated processes with mechanisms to manage resources such as CPU and memory. JX [29] implements a microkernel for managing hardware resources and uses the JVM as the unit of protection domains, running applications and services in different domains to achieve separation of privilege. However, because these safe languages rely on garbage collection, which can introduce unpredictable latencies, they conflict with the real-time requirements of some OS services [24, 50].

The second phase began with the emergence of Rust, a language that does not rely on garbage collection and thus enables high-performance systems programming. Representative kernels include Tock [52, 71], RedLeaf [66], Theseus [10], and Asterinas [67, 68]. The first version of Tock [52] aimed to provide isolation for low-end embedded applications, and now it provides capabilities to restrict access to privileged APIs [71]. RedLeaf captures inter-domain faults to achieve fault isolation. Theseus eliminates state dependencies [11] across domains. Asterinas introduces a framekernel that encapsulates the unsafe Rust required for kernel development, thereby minimizing the kernel’s TCB.

3.2 Heavy Overhead of Current Microkernel Capability Systems

Conventional microkernel capability systems rely on hardware isolation to ensure capability unforgeability, which imposes a significant performance overhead on microkernels. First, capability operations must be delegated to the kernel for execution via system calls, thereby introducing mode-switching overhead. Second, in the multi-server scenarios of microkernels, if a capability operation involves multiple protection domains, it also entails the cost of address-space switching. To avoid these overheads, current safe language-based kernels do not adopt the traditional microkernel’s explicit, capability-centric authority model. Although this approach reduces the performance overhead associated with capabilities, it weakens the capability system’s advantages in areas such as the PoLP.

Safe language-based single-address-space microkernels provide a new opportunity to re-examine the tradeoff between performance and security. First, object capabilities

in safe languages no longer rely on hardware isolation to guarantee their unforgeability [12, 20, 32], which establishes the prerequisite for constructing a capability system within a single address space. Second, the SAS design eliminates the costly CPU mode and context switches involved in resource access. Furthermore, since protection domains and the kernel reside in the same address space, capabilities are invoked through typed interfaces, eliminating the overhead of decoding raw bytes to determine the capability type and the method being called. Therefore, safe language-based microkernels can achieve the security benefits of a capability system without incurring a high overhead.

3.3 New Minimality Requirement for Safe Language-Based OSes

Minimality is the core principle of microkernels. In safe language-based microkernels, however, minimality entails another requirement besides minimizing kernel functionality: unsafe code usage must be minimized. This is because the prerequisite for the isolation provided by a safe language is that the code within a protection domain must be implemented using pure safe code. Otherwise, the type-safe guarantees of the language could be compromised. Consequently, any component containing unsafe code inherently becomes the system’s TCB. Therefore, safe language-based microkernels must not only retain minimal hardware resource abstractions, but also design these abstractions as safe abstractions with sufficiently rich semantics. This enables protection domains residing in the same address space, particularly OS services such as file systems and device drivers, to be implemented entirely in safe Rust.

Fortunately, this presents a natural opportunity. The hardware resources that a microkernel must manage are precisely where unsafe code is most concentrated in a Rust OS. Therefore, these two requirements can be unified by encapsulating hardware resources as microkernel-style kernel objects, centralizing them within a minimal kernel that contains the unsafe code, and providing them to protection domains via safe interfaces. Furthermore, modern microkernels, such as seL4, demonstrate that high expressiveness can coexist with strict minimality, enabling the construction of many practical OSes based on a small TCB (on the order of 10k LoC) [26, 28, 46, 69, 72, 73, 81, 85].

Asterinas has already begun exploring the separation of safe and unsafe code within the intra-kernel address space. By encapsulating unsafe hardware operations within a privileged framework, it constructs the kernel’s memory-safety TCB in a principled manner. Asterinas aims to guarantee the memory-safety property of the entire kernel in a minimal way, thereby answering which unsafe operations must remain within the trusted framework. However, it still retains certain OS functionalities within the TCB, such as the physical memory manager, and it does not use capabilities for the unified granting, derivation, and revocation of authority over

all system resources. Consequently, its privileged framework resembles an expressive safe OS development kit more than a microkernel constrained by principles such as minimality and PoLP.

4 Design

4.1 Overview

safeL4 is a safe language-based microkernel (see the right side of Figure 1). The safeL4 microkernel abstracts system resources as kernel objects and encapsulates them with capabilities. A protection domain uses a Rust crate (a Rust compilation unit) as an isolation boundary. All protection domains run in the same address space and at the same privilege level (kernel mode) as the microkernel. safeL4 requires that protection domains be implemented in pure safe code, and that their use of hardware resources and IDC through the capabilities. safeL4 supports two types of applications: safe tasks and user-level tasks. Safe tasks are OS services or applications that are written purely in safe code and run within protection domains. User-level tasks run in user space and are isolated by hardware, supporting legacy applications written in unsafe languages that invoke OS services within protection domains via POSIX system calls.

safeL4 builds a capability system based on Rust smart pointers, enforcing fine-grained access control over all system resources. safeL4 adheres to the minimality principle of microkernels by adopting a microkernel-like approach that provides low-level abstractions of hardware resources and by providing safe abstractions for non-privileged instructions and the basic data structures that must be implemented with unsafe Rust. These abstractions ensure that protection domains can implement their logic entirely in pure safe code. Furthermore, safeL4 confines a domain’s failures within itself through fault isolation.

Threat model. We describe the threat model of the OS built on safeL4. (1) We trust the Rust compiler to enforce language safety, including memory safety, type safety, and the absence of data races for code written in safe Rust. (2) We trust the Rust core and alloc libraries. Although these libraries contain unsafe code, trusting them is justified due to extensive formal verification efforts on the Rust language [2–4, 19, 39, 49, 51, 58] and its standard library [27, 43]. For example, the RustBelt project [43] has formally verified several Rust libraries containing unsafe code, such as Arc, Rc, Cell, RefCell, Mutex, and RwLock. (3) We trust the implementation of a global heap allocator, which is required to support on-heap containers provided by alloc, such as Vec and BTreeMap. (4) The TCB of a safeL4-based OS includes the safeL4 microkernel and its dependencies. (5) All code within protection domains is excluded from the TCB.

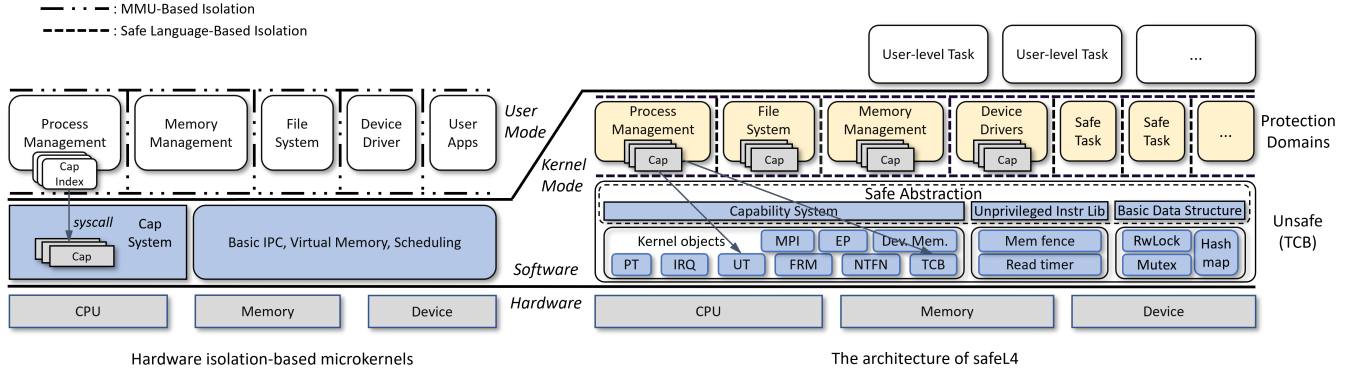


Figure 1. Comparison between hardware isolation-based microkernels and safeL4 (EP: Endpoint; Dev. Mem.: DeviceMemory; PT: PageTable; IRQ: Interrupt; UT: Untyped; FRM: Frame; NTFN: Notification).

4.2 Capability System

The capability system encapsulates safe abstractions of system resources (kernel objects, which are described in §4.3.1) and serves as the interface that safeL4 exposes to protection domains.

4.2.1 Capability System Design. In Rust, the combination of Arc and Weak smart pointers is a common pattern for shared ownership. Its semantics fit the basic sharing requirements of capabilities, and many Rust-based operating systems use this native mechanism to implement capabilities for shared resources. However, for a strict capability system, such as one that supports the take-grant model [34, 57], directly using Arc and Weak as capabilities introduces the following two issues.

Issue 1: Arc/Weak cannot support revocation semantics. In a capability system, subjects must be able to revoke resources they have shared. However, the semantics of Arc prevent such revocation. Once subject A duplicates an Arc and passes it to subject B, A cannot perform a revocation as long as B still holds that Arc. Additionally, the fact that Arc/Weak cannot represent multi-level capability derivation hierarchies prevents revocation. For example, suppose domain A owns a kernel object through an Arc and shares it with domain B as a Weak, and domain B then shares it with domain C. Arc/Weak cannot track the derivation relationship between domains B and C. As a result, in the sharing chain “A→B→C”, domain B cannot revoke domain C’s access to the kernel object.

Issue 2: Weak::upgrade prevents revocation. When a domain calls Weak::upgrade(), it obtains an Arc and can hold the corresponding kernel object indefinitely. While this behavior is acceptable in common Rust programs, it is problematic in a capability system: if a protection domain can hold the upgraded Arc indefinitely, the original resource owner cannot revoke it.

To address the two issues, we combine Rust’s Arc/Weak with a *capability derivation tree* (CDTree) and propose the

capability system as follows (Listing 1). ❶ Each kernel object is wrapped in an `Arc<RwLock<KObj>>`, called `RawCap`, which ensures safe concurrent access to the object. ❷ The capability system does not expose `RawCap` directly to protection domains. Instead, it provides a wrapper called `Cap`. `Cap` is a public struct visible to protection domains, but its internal fields are marked `pub(crate)`, restricting their visibility only to the microkernel. ❸ The microkernel maintains the CDTree, a private global tree composed of `CdtNode`, to record derivation relationships among capabilities. For all capability operations (e.g., derive, mint, revoke), the capability system maintains the following invariant: each node (`CdtNode`) in the CDTree has a 1:1:1 correspondence with a `RawCap` and its associated `Cap`. ❹ The microkernel exposes a procedural macro that allows protection domains to invoke methods of the kernel object by a capability. ❺ Each `Cap` contains a private field `rights` that specifies its authorities.

```
1 pub(crate) struct RawCap {
2     pub(crate) object: Arc<RwLock<KObj>>,
3 }
4 pub(crate) struct CdtNode {
5     pub(crate) cap: Arc<RawCap>,
6     pub(crate) child: Vec<Arc<Mutex<CdtNode>>>,
7 }
8 pub struct Cap {
9     pub(crate) raw_cap: Weak<RawCap>,
10    pub(crate) rights: Rights,
11    pub(crate) cdt_node: Weak<Mutex<CdtNode>>,
12 }
```

Listing 1. Struct definitions in safeL4’s capability system.

For **Issue 1**, our solution is to enforce revocation semantics by maintaining the derivation relationships when resources are shared via Arc/Weak. We bind CDTree maintenance to the Cap operations built on Arc/Weak, ensure the CDTree retains ownership of the RawCap, and restrict domains to holding only a Weak to the RawCap through the Cap. Figure 2 shows the changes in the CDTree when performing capability derivation and revocation operations.

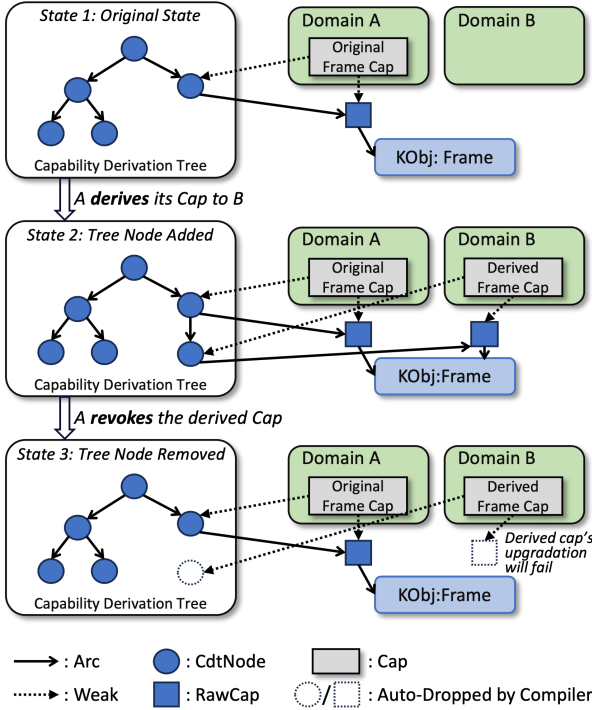


Figure 2. Derivation and revocation of the capability system.

State 1 → *State 2*: When domain A derives a Cap for domain B, the capability system clones the RawCap and creates a new CdtNode to hold it. It then uses this CdtNode and the cloned RawCap to create a new Cap, returning this new Cap to domain A. Domain A can then share the derived Cap with domain B.

State 2 → *State 3*: When domain A revokes the Cap, the capability system locates the CdtNode via the Cap and drops the derived CdtNode. Consequently, the RawCap owned by the derived CdtNode is dropped as well. Thereafter, although domain B still holds the derived Cap, when it attempts to invoke the kernel object through this Cap, upgrading the Weak to the RawCap will fail.

For **Issue 2**, our key insight is that the capability system must limit the scope of Weak upgrades, and this enforcement must be guaranteed by the microkernel. safeL4 enforces this by making the RawCap visible only within the microkernel crate using `pub(crate)`, preventing protection domains from directly upgrading the Weak reference inside a Cap. safeL4 provides a procedural macro interface that redirects the kernel object method invocation to a public proxy function, which upgrades a Cap’s inner Weak (see §5.3 for details). This ensures that all Weak upgrades occur only within the proxy function; thus, the lifetime of the upgraded Arc, and consequently the duration of resource access, is strictly controlled by the microkernel.

4.2.2 Security Analysis of the Capability System.

Capability unforgeability. In safeL4, protection domains

are restricted to pure safe Rust, meaning they can only create capabilities through the capability system’s public APIs. While safe Rust allows creating a raw pointer and performing pointer arithmetic (e.g., using `ptr::wrapping_add`) to point to a capability or other sensitive data, an attacker cannot exploit these operations to break the safety enforced by safe Rust. Any attempt to dereference such a raw pointer would require unsafe code blocks, which are disallowed in protection domains. Moreover, all interfaces of the safeL4 microkernel are safe and do not accept raw pointers as parameters. Therefore, attackers cannot convert a forged raw pointer into a valid capability through safeL4’s interfaces.

Protection domains cannot bypass the capability system. The pointer to the kernel object within a capability is encapsulated as a private field, preventing direct dereferencing in protection domains. In addition, each kernel object is defined as a private structure within the microkernel, ensuring that protection domains cannot instantiate or directly access kernel objects, nor can they directly call their methods to bypass the authority check. Consequently, the procedural macro provided by the microkernel is the only interface through which protection domains can invoke operations on the kernel objects referenced by capabilities.

4.3 The Minimality Design and Our Advantages

In the context of safe language-based microkernels, the minimality principle requires not only that the microkernel provide the minimum functionality, but also that it provide safe abstractions for all necessary unsafe operations, thereby ensuring that protection domains can be implemented exclusively in pure safe Rust.

4.3.1 The Minimality Design. To maintain minimality, safeL4 provides low-level safe abstractions for the following types of unsafe operations.

Low-level safe abstraction for system resources. The first design principle for maintaining kernel minimality in safeL4 is to adopt the microkernel approach to constructing low-level abstractions for system resources. Inspired by seL4, we abstract fundamental hardware resources (CPU, memory, and I/O), privileged instructions, and communication functionality into corresponding kernel objects, and expose their interfaces as object methods. safeL4 requires the method signatures of kernel objects to be strictly safe, while their implementations may use unsafe code.

For CPUs, the TCB (*thread control block*) object encapsulates access to and modification of CPU registers. Physical memory is abstracted as the Frame object, which provides safe methods for accessing memory. *Memory-mapped I/O* (MMIO) devices are represented as DeviceMemory objects that provide safe methods for volatile read and write operations on device registers. Interrupt events from peripheral devices are abstracted as Notification objects. For privileged instructions, safeL4 follows seL4’s approach by

wrapping them in the methods of kernel objects. OS services invoke these privileged instructions through these methods. Since the scheduler is moved out of the microkernel, safeL4 introduces a new kernel object, MPI (*multiprocessor information*), to provide access to CPU identifier registers. We design the Endpoint object to handle synchronous communication, and use Notification objects for asynchronous communication.

Safe abstraction for non-privileged instructions. The Rust core library provides wrappers for a subset of non-privileged instructions. For non-privileged instructions not covered by core that are required in device driver development, such as timer read and memory barrier instructions, safeL4 encapsulates them as safe functions in a library crate for protection domains.

Safe abstraction for basic data structures. Some fundamental data structures commonly used in Rust programs require unsafe implementations, such as synchronization primitives (e.g., Mutex, RwLock) and on-heap containers (e.g., HashMap, LinkedList). The microkernel encapsulates these basic data structures and provides safe interfaces to protection domains.

4.3.2 Our Advantages. Our minimal design offers the following advantages.

Removing CNode. In seL4, capabilities must be stored within the kernel to ensure they cannot be forged. A specialized kernel object type, *CNode*, is required to contain the capabilities of each protection domain. Such a design forces all capability transfers to go through the microkernel, introducing system-call overhead. For example, in Google Cantrios (an seL4-based OS) [30], a temporary CNode called *recv_cnode* is responsible for receiving and transferring Frame capabilities, requiring at least four system calls per memory allocation. In contrast, safeL4 stores capabilities directly within each domain and can transfer them as parameters. This design is secure because safeL4 leverages Rust to guarantee that capabilities cannot be maliciously forged or duplicated by domains. Consequently, capability operations such as moving, delegating, and bulk sharing, which in seL4 are implemented as system calls, can be implemented in safeL4 through IDC with capabilities as parameters.

Removing the scheduler from the microkernel. The minimality principle requires the microkernel to be policy-free. However, the L4 family of microkernels has traditionally struggled to move the scheduler out of the kernel due to intolerable performance overhead. In safeL4, since IDC achieves performance comparable to that of function calls, both the scheduling queue and scheduling policies are moved to the process management domain, while the microkernel handles only context switching.

4.4 Fault Isolation

Because the safe language-based microkernel and its protection domains share a single address space, a panic in one domain will propagate and crash the entire OS. Therefore, safeL4 must provide a fault isolation mechanism to capture panics permitted by the safe language, such as Rust’s panics.

Inspired by some Rust-based OSes [10, 66, 68], we implement a lightweight fault isolation mechanism. When a protection domain invokes another domain via a synchronous IDC, safeL4 inserts fault isolation logic at the invocation point using a Rust procedural macro (see §5.2 for details). If a thread panics within the callee domain, the failure is captured at the call site. safeL4 unwinds the stack frames in the callee domain and cleans up the associated resources. The caller domain then receives an error from the IDC.

5 Implementation

5.1 The safeL4 Microkernel

The safeL4 microkernel provides system resource operations to protection domains by implementing a set of kernel objects, including TCB, Untyped, DeviceMemory, Frame, PageTable, MPI, Notification, Interrupt, and Endpoint. All these kernel objects are accessed via capabilities. In addition, the microkernel provides safe wrappers for non-privileged instructions and basic data structures.

5.2 Inter-Domain Communication

safeL4 supports both asynchronous and synchronous IDC. The asynchronous IDC is built on the Notification object. In addition to supporting the asynchronous semantics of seL4’s Notification (e.g., signal, wait, and poll), safeL4 extends it with a message-passing mechanism based on zero-copy ownership transfer, enabling efficient asynchronous communication between domains.

We implement synchronous IDC based on the Endpoint object. The callee domain uses link-time registration to register its interface with the microkernel, and then Endpoint uses the registered interface to invoke the callee domain.

5.2.1 Link-Time Registration for Callee Crate. Since the microkernel crate is a dependency of all protection domains, it cannot directly reference any domain crate. Otherwise, it would lead to a circular dependency.

To resolve this, we modify the linking phase of the microkernel and protection domains. Each callee crate uses `trusted_kernel_export!`, a Rust attribute macro provided by the microkernel, to annotate the functions it intends to export to other domains and the microkernel (Listing 2). During linking, the linker aggregates all exported interfaces from callee crates into a designated section of the compiled image. Each interface can be identified by a unique key consisting of the callee’s crate name and its function signature (including function name, parameter types, and return type).

When loading the callee image, the microkernel parses this section to obtain the interfaces of the callee and stores their function pointers for later invocation.

```
1 // in MemoryManager(callee crate)
2 #[trusted_kernel_export]
3 pub fn ut_alloc_frame() -> Result<Cap, UtError> {
4     UT_MANAGER.lock().do_alloc::<Frame>(FRAME_SIZEBIT)
5 }
6 // in ProcessManager(caller crate)
7 fn alloc_frame_cap() -> Result<FrameCap, UtError> {
8     trusted_kernel_invoke!(mm::ut_alloc_frame() ->
9         Result<Cap, UtError>).unwrap()
10 }
```

Listing 2. Example code of registering an interface.

```
1 // trusted_kernel_invoke! calls this function after
2   the type-match check
3 pub fn invoke_proxy<T>(
4     api_identifier: ExportedAPIIdentifier,
5     arg: *const (),
6     ret: *mut (),
7 ) -> Option<T> {
8     GLOBAL_API_RESERVOIR.read()
9     // find callee interface according to identifier
10    tuple
11    .get(&api_identifier)
12    .and_then(|interface| unsafe {
13        // wrap callee execution
14        crate::catch_unwind(|| {
15            interface.transmute_then_invoke(arg, ret);
16        })
17        .ok()
18        .and_then(|_| {
19            let ret_place_holder = &mut *(ret as *mut
20                Option<MaybeUninit<T>>);
21            ret_place_holder.take().map(|inner| inner.
22                assume_init())
23        })
24    })
25 }
```

Listing 3. Simplified safeL4 microkernel workflow for resolving and calling IDC interfaces.

5.2.2 Runtime IDC Invocation. An Endpoint uses the procedural macro `trusted_kernel_invoke!` (Listing 3) to invoke the callee domain’s interface. The macro first performs a type matching to ensure that the parameters passed by the caller crate conform to the requirements of the callee interface. It then directly locates the corresponding callee interface using the compile-time generated unique key. Before transferring control to the callee domain, the invocation is wrapped in a `catch_unwind` block to capture and handle potential panics in the callee domain. Additionally, safeL4 uses a custom compilation target to generate a section in the OS image for storing exception-handling information. Leveraging this information, safeL4 employs the unwinding library [31] to implement a stack-unwinding mechanism.

5.3 Capability Invocation

A protection domain holds capabilities pointing to different kernel objects, and all accesses to a given kernel object must be performed by invoking the corresponding Cap. In safeL4, such capability invocation is exposed through `kobj!`, a procedural macro provided by the microkernel. The macro takes Cap as input and redirects the kernel object method to a public proxy function, which then performs the following steps: (1) checks authority, (2) temporarily upgrades the Weak to the corresponding RawCap, and (3) tries to obtain the RLock guarding the KObj inside the RawCap. The proxy function then executes the target method of the kernel object. After invocation, the proxy function releases the acquired RLock, exits the scope of the Weak: : upgrade, and returns.

5.4 OS Services in Protection Domains

We implemented fundamental OS services on safeL4, including process management, memory management, a file system, and several peripheral drivers.

5.4.1 Process Management and Memory Management.

The *process management* (PM) is responsible for creating, deleting, and scheduling all threads in the OS. PM uses the TCB object to implement the thread abstraction for both safe tasks and user-level tasks, and implements a scheduler. In the multicore scheduling implementation, PM obtains the hardware core ID via the capability of an MPI object.

During system startup, the safeL4 microkernel initializes the physical memory reserved for protection domains as Untyped objects and transfers the corresponding capabilities to the memory management domain. The memory management domain implements a buddy system and a slab system based on these Untyped objects, and provides interfaces for memory allocation and deallocation to other domains.

5.4.2 Porting RedoxFS. RedoxFS [83] is a file system inspired by ZFS [9] that runs as a user-level process on Redox OS [84]. We port RedoxFS and run it in a protection domain on safeL4, and the interfaces of RedoxFS are registered using `trusted_kernel_export!`.

We remove all unsafe code blocks from the original RedoxFS code by leveraging our safe abstractions. RedoxFS performs in-place serialization and deserialization of inode data structures, which requires unsafe type conversions in Rust. To address this, inspired by PODType [82], we introduce a dedicated unsafe trait named `Convert` within the microkernel to support controlled type conversion and make it available for protection domains. We further enforce strict semantic constraints on this trait to restrict its behavior, which provides the following three additional guarantees.

- (1) Types that implement this trait cannot contain any function or data pointers, and all fields must be owned values.

- (2) Types that implement this trait can only be converted to and from a byte array.
- (3) The trait ensures that all type conversions occur in place with bound-checking to prevent out-of-bounds memory access during conversion.

Since this trait is marked as unsafe within the microkernel, protection domains cannot implement it directly. Instead, protection domains use the `#[derive(Convert)]` macro to mark their data structures that require the trait, and let the Rust compiler check if the requirements are satisfied and automatically derive implementations for these structures.

5.4.3 Device Drivers. The interrupt dispatch routine remains within the microkernel, while peripheral device drivers are implemented in protection domains. We have ported Redox’s SD card driver and implemented drivers for UART, timer, and DMA. All device drivers access MMIO registers through the capabilities of their `DeviceMemory` objects, each of which represents a specific device memory region.

To support a new peripheral driver, we can create a `DeviceMemory` object pointing to the device’s MMIO region, pass the corresponding capability to the device driver’s protection domain during driver initialization, and bind the device’s interrupt (an `Interrupt` object) to the driver’s interrupt-handling IDC interface. For example, when porting Redox’s SD card driver, we replace its original unsafe operations, namely direct register access using `ptr::read_volatile` and `ptr::write_volatile`, with safe invocations on the capability of the `DeviceMemory` object (Listing 4).

```

1 // Redox's unsafe way to access MMIO area.
2 unsafe {(start as *mut T).add(offset).write_volatile(
3     val);}
4 // safeL4's cap-gated access (Offset is checked)
5 let result = kobj!(<device_memory_cap as
6     DeviceMemoryCap>.write(offset, val)
7     .map_err(|_| DMErrror::DMWriteError));

```

Listing 4. safeL4 replaces unsafe MMIO access with safe invocations on the capability of the `DeviceMemory` object.

5.4.4 Musl Libc Support. To support user-level applications written in unsafe languages, we port musl libc to safeL4’s user layer and implement 50 system calls in OS services. A system call invoked by musl libc is transferred to the corresponding OS services.

6 Evaluation

We evaluate the code and TCB size of safeL4, and aim to achieve the following evaluation objectives.

- **Objective 1.** By comparing safeL4 with seL4, evaluate the performance differences in low-level operations (such as IPC, interrupt handling, signal, and scheduling) between a safe language-based microkernel and a hardware isolation-based microkernel.

- **Objective 2.** Evaluate the performance differences between safeL4, seL4, and Linux when handling realistic workloads.
- **Objective 3.** Evaluate the file system-related microbenchmark performance between safeL4 and Redox, to reveal the impact of different microkernel architectures.

6.1 Experiment Setup

All experiments are conducted on a Raspberry Pi 3B+ board with four Cortex-A53 cores (1.4 GHz), 1 GB SDRAM, and a 32 GB SD card. We have set up the following OSes on this hardware platform to achieve our evaluation objectives.

seL4. seL4 is a hardware isolation-based microkernel with the fastest IPC. We evaluate the performance differences between safeL4 and seL4 in terms of their low-level abstraction of hardware resources (*Objective 1*) and realistic workloads (*Objective 2*). The most common deployment scenario for seL4 is as a library OS (LibOS), in which resources are statically allocated, and most computations do not require system calls. We run YCSB benchmarks [18] on SQLite3 [78] in the user space of safeL4 and seL4 LibOS.

Redox. Redox OS is a relatively mature microkernel-based general-purpose OS based on hardware isolation and implemented in Rust. We have ported RedoxFS and the SD card driver to safeL4’s protection domains while maintaining source code consistency. We run file system-related microbenchmarks in user space on both Redox and safeL4 to evaluate the performance difference between safe language-based isolation and hardware-based isolation (*Objective 3*).

Linux. Linux is a representative monolithic OS. We use Ubuntu 22.04 (Linux kernel 6.6.12) with the ZFS file system. We run SQLite3 in the user space of safeL4 and Linux (*Objective 2*).

6.2 TCB Evaluation

The entire safeL4 OS consists of 22,018 LoC (Table 3). The microkernel contains 11,688 LoC, including 1,765 lines of unsafe code, which accounts for 15.1% of the total codebase. We support the RISC-V64 and AArch64 architectures. Table 3 shows that the amount of unsafe code required to support a new CPU architecture is quite small, 462 LoC for AArch64 and 349 LoC for RISC-V64, respectively.

Table 3. Lines of code for safeL4.

	LoC	Crate	LoC	Crate	LoC
Safe code	9 923	Microkernel	11 688	DMA	328
Unsafe code:		Process management	3 943	UART	261
Arch-independent	954	Memory management	502	RAM Disk	110
AArch64	462	RedoxFS	4 254	Mailbox	81
RISC-V64	349	SD card driver	777	Timer	74
Total (Microkernel)	11 688	Total			22 018

We evaluate safeL4’s TCB and how a safe language affects the TCB code size. The TCB is determined according to our threat model, primarily including the Rust core and alloc libraries, and any crates containing unsafe code that the

entire OS depends on (including those used by protection domains), along with their direct or indirect dependencies.

To ensure accurate TCB measurement, we do not directly count the source lines of TCB crates, since not all code in the codebase is linked into the final image. Instead, we adopt the *linked code size* (LCS) methodology used by Asterinas [68], i.e., the actual number of source lines linked during compilation. We count all source lines referenced in the linked IR, which is generated by the LLVM toolchain when compiling the microkernel, its dependencies, and the OS service crates.

The evaluation results (Table 4) show that (1) in Rust-based OSes, the Rust core and alloc libraries occupy a large part of the TCB. (2) Compared with other Rust-based OSes, safeL4’s low-level abstraction approach yields the smallest TCB for system resource abstraction (others in Table 4). Although Tock also has a small TCB in this aspect, it is a lightweight OS aimed at low-power microcontrollers.

Table 4. Comparison of TCB Size (unit: LCS).

OS	TCB		Non-TCB	Total
	core + alloc	others		
Asterinas	9 442	11 090	58 165	78 697
Theseus	8 720	40 459	9 595	58 774
RedLeaf	4 357	17 240	4 813	26 410
Tock	1 911	2 903	1 814	6 628
safeL4	6 248	2 720	6 081	15 049

6.3 Microbenchmark Comparison with seL4

We use seL4bench [74] as a microbenchmark suite to compare the performance of safeL4 and seL4. seL4bench is designed to evaluate various low-level operations of the seL4 microkernel. On safeL4, we reimplement all seL4bench benchmarks in Rust, excluding VCPU (virtualization), and execute them in protection domains.

The evaluation results (Table 5) show that safeL4 is 1.91× (geometric mean) faster on average than seL4, with IDC demonstrating the most significant performance advantage, being 7.8× to 22.3× faster than seL4. The results may help answer the question about the performance comparison between safe language-based microkernels and those that rely on hardware isolation [35]: for microkernels, the overhead introduced by Rust, a safe systems programming language (including type safety checks and fault isolation mechanisms), is lower than the overhead introduced by hardware isolation (including CPU context switching, privilege-level switching, and page table switching).

Synchronization-related benchmarks (signal/schedule/sync) also benefit from safeL4. In safeL4, the primary overhead of a synchronization operation comes from accessing CPU contexts through capabilities, CPU context switching triggered by thread switches, and atomic operations within semaphores. They do not involve CPU mode switching or address space switching, thereby outperforming seL4.

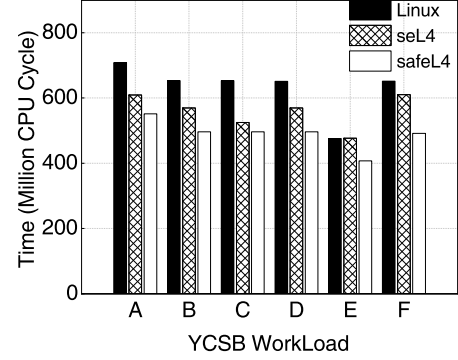


Figure 3. SQLite3 YCSB benchmark, smaller is better.

Here, we explain why safeL4 underperforms seL4 in the frame allocation benchmark. This primarily stems from design divergences in concurrency strategies of kernel data structures. seL4 employs coarse-grained concurrency control via a big kernel lock, which ensures exclusive access, allowing kernel data structures to operate without internal synchronization checks. In contrast, safeL4 adopts a fine-grained locking design. Accessing kernel objects via capabilities in safeL4 requires acquiring a read-write lock (RwLock) to ensure correctness in concurrent environments, which inevitably introduces synchronization overhead at the granularity of kernel objects. We profile the synchronization overhead incurred by this fine-grained locking (Table 7). The results indicate that during frame allocation, the initialization of a capability with RwLock and Arc consumes an average of 656.11 CPU cycles, constituting the dominant overhead. Furthermore, a portion of the frame allocation overhead in safeL4 originates from maintaining the CDTTree.

6.4 Macrobenchmark Comparison with seL4 and Linux

We evaluate the performance of safeL4, seL4, and Linux using the YCSB benchmark on SQLite3. SQLite3 is ported to seL4 LibOS and deployed as a user-level task on safeL4. The workload comprises six standard YCSB scenarios, covering read, write, update, and mixed operations on a table containing 1,000 records.

The results (Figure 3) show that safeL4 outperforms seL4 by 14.3% and Linux by 23.2% on average. Both safeL4 and the seL4 LibOS achieve higher performance than Linux.

To explain safeL4’s advantage over Linux, we profile system calls during the YCSB benchmark. Four system calls, `fcntl`, `newfstatat`, `pread64`, and `pwrite64`, account for more than 70% of the total execution time. The system calls implemented by safeL4 are simpler than Linux’s. For example, there is much less locking logic (no byte-range locking and no need for a global lock linked list), fewer system states to maintain, no access control checks, and so on.

In this benchmark, the performance improvement of safeL4 over seL4 is not as good as in seL4bench, for the following

Table 5. The seL4bench results in CPU cycles, smaller is better.

seL4bench	Description	seL4	safeL4	Perf*
Fault	hardware fault triggering-to-handling	589.470 ± 22.487	309.952 ± 5.206	1.902
Hardware	calling into kernel and return	149.000 ± 34.795	44.000 ± 0.000	3.386
IPC/IDC	invoke a server in a different address space (seL4)/domain (safeL4)	249.000 ± 0.000	32.000 ± 0.000	7.781
	a server replies to a client in a different address space (seL4)/domain (safeL4)	268.000 ± 0.000	12.000 ± 0.000	22.330
IRQ	invoke an interrupt handler in a user-thread (seL4)/domain (safeL4)	622.440 ± 21.835	413.632 ± 1.502	1.505
Page mapping	prepare one page table	1 354.438 ± 120.572	1 153.889 ± 151.577	1.174
	map one page	390.377 ± 4.420	260.900 ± 12.518	1.496
	allocate one frame capability	538.345 ± 24.992	949.200 ± 14.119	0.567
Signal	asynchronous signal delivery	406.050 ± 22.614	113.684 ± 1.243	3.572
	synchronous signal delivery	560.830 ± 22.979	449.309 ± 8.072	1.248
Scheduler	yield a thread	382.310 ± 19.130	400.056 ± 0.631	0.956
	schedule between two threads	758.100 ± 15.400	584.955 ± 3.462	1.296
Synchronization	broadcast a signal through notification	1 370.860 ± 26.950	892.400 ± 23.589	1.536
	the last waiter of a waiting chain being waked	71 296.000 ± 14.949	45 868.900 ± 208.884	1.554
	producer to consumer latency	1 217.740 ± 140.721	876.600 ± 9.088	1.389
	consumer to producer latency	1 194.160 ± 28.889	860.800 ± 3.513	1.387
SMP	inter-core IPC round-trip	6 888.610 ± 0.410	4 877.980 ± 2.460	1.411
Geometric Mean				1.910

* Normalized performance, use seL4/safeL4 to indicate safeL4's performance advantage over seL4. Higher is better.

Table 6. File system-related comparison with Redox.

LMBench	Description	Unit	Redox	safeL4	Perf*
memory map	map then unmap a block of memory	μs	27.789 ± 0.049	21.795 ± 0.052	1.275
page fault	triggers page fault for each page	μs	150.988 ± 0.129	94.336 ± 0.157	1.601
file access	open, read then close a file	MB/s	8.269 ± 0.053	18.142 ± 0.009	2.194
	read a file	MB/s	25.199 ± 0.139	55.618 ± 0.007	2.207
memory map	map, read, then unmap a file	MB/s	345.429 ± 7.180	380.940 ± 0.000	1.103
	open, map, read, unmap, close	MB/s	8.287 ± 0.001	12.082 ± 0.002	1.458
file create/delete	create files	files/s	32.685 ± 1.960	40.894 ± 0.986	1.251
	delete files	files/s	74.854 ± 4.370	82.138 ± 1.833	1.097
Geometric Mean					1.470

* Normalized performance. For throughput, use safeL4/Redox; for latency, use Redox/safeL4. Higher is better.

Table 7. Profile of allocation of a frame.

Operation	Avg. Cycles
Acquire the RwLock on a capability to access Untyped kernel object	99.28
Retype a Frame kernel object from the Untyped kernel object	165.32
Create a Frame capability using the Frame kernel object	656.11
Insert the Frame capability's CdtNode to its parent	29.00

reasons. First, SQLite3 and YCSB, as implemented in unsafe language, run in user space on safeL4, so they need to allocate resources via system calls. Second, the seL4 LibOS adopts a static allocation strategy: the memory required by applications is allocated in one shot during initialization, reducing the number of system calls during execution. Third, in the seL4 LibOS, SQLite3, the file system and drivers run in the same address space, and there is no IPC overhead between them. The second and third reasons also explain why seL4 outperforms Linux in this benchmark.

6.5 Microbenchmark Comparison with Redox

We port the RedoxFS and SD card driver from Redox OS, maintaining source and semantic consistency. In Redox OS, the file system and drivers run as user-level processes. In safeL4, they run in protection domains.

We cannot run LMBench directly on Redox because it only supports a limited set of POSIX system calls on AArch64. We implement an LMBench subset related to the file system in Rust and run it in user space on both safeL4 and Redox.

The results (Table 6) show that safeL4 outperforms Redox in all file system-related microbenchmarks, with a geometric-mean speedup of 1.47×. This experiment demonstrates that it is feasible to port existing typical OS services to safeL4 using its safe abstractions and that these OS services can benefit from safeL4's single-address-space design.

7 Conclusion

This paper presents safeL4, a safe language-based microkernel built around a capability-centered design. The key idea is to combine a single-address-space design for low-overhead capability invocation with safe language-enforced capability unforgeability, and to treat the capability as a first-class kernel abstraction for uniformly exposing system resources to protection domains. Our evaluation shows that safeL4's primitives achieve 1.91× the performance of seL4 on average, and that although the Rust toolchain accounts for a notable portion of the TCB, the kernel itself remains compact.

References

- [1] Mike Accetta, Robert Baron, William Bolosky, David Golub, Richard Rashid, Avadis Tevanian, and Michael Young. 1986. Mach: A new kernel foundation for UNIX development. In *Proceedings of the 1986 USENIX Summer Conference*. USENIX Association, 93–113. <https://eecs582.github.io/readings/mach-usenix86.pdf>
- [2] Vytautas Astrauskas, Aurel Bily, Jonáš Fiala, Zachary Grannan, Christoph Matheja, Peter Müller, Federico Poli, and Alexander J Summers. 2022. The Prusti project: Formal verification for Rust. In *Proceedings of the 14th International NASA Formal Methods Symposium (NFM)*. Springer, 88–108. doi:10.1007/978-3-031-06773-0_5
- [3] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J Summers. 2019. Leveraging Rust types for modular specification and verification. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 147:1–147:30. doi:10.1145/3360573
- [4] Sacha-Élie Ayoun, Xavier Denis, Petar Maksimović, and Philippa Gardner. 2025. A hybrid approach to semi-automated Rust verification. *Proceedings of the ACM on Programming Languages* 9, PLDI (2025), 970–992. doi:10.1145/3729289
- [5] Godmar Back, Wilson C Hsieh, and Jay Lepreau. 2000. Processes in KaffeOS: Isolation, Resource Management, and Sharing in Java. In *Proceedings of the 4th Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 333–346. <https://dl.acm.org/doi/epdf/10.5555/1251229.1251252>
- [6] Brian N Bershad, Craig Chambers, Susan Eggers, Chris Maeda, Dylan McNamee, Przemyslaw Pardyak, Stefan Savage, and Emin Gün Sirer. 1994. SPIN: An extensible microkernel for application-specific operating system services. In *Proceedings of the 6th ACM SIGOPS European Workshop: Matching Operating Systems to Application Needs (EW)*. ACM, 68–71. doi:10.1145/504390.504408
- [7] Brian N Bershad, Stefan Savage, Przemyslaw Pardyak, Emin Gün Sirer, Marc E Fiuczynski, David Becker, Craig Chambers, and Susan Eggers. 1995. Extensibility safety and performance in the SPIN operating system. In *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 267–284. doi:10.1145/224056.224077
- [8] Alan C Bomberger, Norman Hardy, A Peri Frantz, Charles R Landau, William S Frantz, Jonathan S Shapiro, and Ann C Hardy. 1992. The KeyKOS® nanokernel architecture. In *Proceedings of the USENIX Workshop on Micro-kernels and Other Kernel Architectures*. USENIX Association, 95–112. <https://flint.cs.yale.edu/cs428/doc/NanoKernel.pdf>
- [9] Jeff Bonwick, Matt Ahrens, Val Henson, Mark Maybee, and Mark Shellenbaum. 2003. *The Zettabyte File System*. Technical Report. Sun Microsystems, Inc. <https://web.mit.edu/6.033/www/papers/zfs.pdf>
- [10] Kevin Boos, Namitha Liyanage, Ramla Ijaz, and Lin Zhong. 2020. The-seus: An experiment in operating system structure and state management. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 1–19. <https://www.usenix.org/system/files/osdi20-boos.pdf>
- [11] Kevin Boos, Emilio Del Vecchio, and Lin Zhong. 2017. A characterization of state spill in modern operating systems. In *Proceedings of the 12th European Conference on Computer Systems (EuroSys)*. ACM, 389–404. doi:10.1145/3064176.3064205
- [12] John Boyland, James Noble, and William Retert. 2001. Capabilities for sharing: A generalisation of uniqueness and read-only. In *Proceedings of the 15th European Conference on Object-Oriented Programming (ECOOP)*. Springer, 2–27. doi:10.1007/3-540-45337-7_2
- [13] Andrew Boyton. 2009. A verified shared capability model. In *Proceedings of the 4th International Workshop on Systems Software Verification (SSV)*. Elsevier, 25–44. doi:10.1016/J.ENTCS.2009.09.058
- [14] Allan Bricker, Michel Gien, Marc Guillemont, Jim Lipkis, Douglas Orr, and Marc Rozier. 1991. A new look at microkernel-based UNIX operating systems: Lessons in performance and compatibility. In *Proceedings of the EuroOpen Spring 1991 Conference*. http://fqrd.fr/publications/c_horus/CS-TR-91-7.pdf
- [15] Anton Burtsev, Dan Appel, David Detweiler, Tianjiao Huang, Zhaofeng Li, Vikram Narayanan, and Gerd Zellweger. 2021. Isolation in Rust: What is missing?. In *Proceedings of the 11th Workshop on Programming Languages and Operating Systems (PLOS)*. ACM, 76–83. doi:10.1145/3477113.3487272
- [16] Luca Cardelli, James Donahue, Lucille Glassman, Mick Jordan, Bill Kalsow, and Greg Nelson. 1992. Modula-3 language definition. *ACM SIGPLAN Notices* 27, 8 (1992), 15–42. doi:10.1145/142137.142141
- [17] Haibo Chen, Xie Miao, Ning Jia, Nan Wang, Yu Li, Nian Liu, Yutao Liu, Fei Wang, Qiang Huang, Kun Li, Hongyang Yang, Hui Wang, Jie Yin, Yu Peng, and Fengwei Xu. 2024. Microkernel goes general: Performance and compatibility in the HongMeng production microkernel. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 465–485. <https://www.usenix.org/system/files/osdi24-chen-haibo.pdf>
- [18] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*. ACM, 143–154. doi:10.1145/1807128.1807152
- [19] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A foundry for the deductive verification of Rust programs. In *Proceedings of the 23rd International Conference on Formal Engineering Methods (ICFEM)*. Springer, 90–105. doi:10.1007/978-3-031-17244-1_6
- [20] Jack B Dennis and Earl C Van Horn. 1966. Programming semantics for multiprogrammed computations. *Commun. ACM* 9, 3 (1966), 143–155. doi:10.1145/365230.365252
- [21] Dong Du, Zhichao Hua, Yubin Xia, Binyu Zang, and Haibo Chen. 2019. XPC: Architectural support for secure and efficient cross process call. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)*. ACM, 671–684. doi:10.1145/3307650.3322218
- [22] Dhammika Elkaduwe, Gerwin Klein, and Kevin Elphinstone. 2008. Verified protection model of the seL4 microkernel. In *Proceedings of the 2nd International Conference on Verified Software: Theories, Tools, Experiments (VSTTE)*. Springer, 99–114. doi:10.1007/978-3-540-87873-5_11
- [23] Kevin Elphinstone and Gernot Heiser. 2013. From L3 to seL4 what have we learnt in 20 years of L4 microkernels?. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 133–150. doi:10.1145/2517349.2522720
- [24] Paul Emmerich, Simon Ellmann, Fabian Bonk, Alex Egger, Esaü García Sánchez-Torija, Thomas Günzel, Sebastian Di Luzio, Alexandru Obada, Maximilian Stadlmeier, Sebastian Voit, and Georg Carle. 2019. The case for writing network drivers in high-level programming languages. In *Proceedings of the 15th ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. IEEE, 1–13. doi:10.1109/ANCS.2019.8901892
- [25] Manuel Fähndrich, Mark Aiken, Chris Hawblitzel, Orion Hodson, Galen Hunt, James R Larus, and Steven Levi. 2006. Language support for fast and reliable message-based communication in Singularity OS. In *Proceedings of the 1st ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys)*. ACM, 177–190. doi:10.1145/1217935.1217953
- [26] Juliana Furgala. 2023. Magnetite: Rust-based OS services for seL4. <https://sel4.systems/Foundation/Summit/2023/slides/day2-02-furgala.pdf>. Presentation slides at the seL4 Summit 2023.
- [27] Lennard Gäher, Michael Sammler, Ralf Jung, Robbert Krebbers, and Derek Dreyer. 2024. RefinedRust: A type system for high-assurance verification of Rust programs. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 1115–1139. doi:10.1145/3656422
- [28] Gapfruit AG. 2025. Gapfruit OS: Secure and scalable OS. <https://www.gapfruit.com/>.
- [29] Michael Golm, Meik Felser, Christian Wawersich, and Jürgen Kleinöder. 2002. A Java operating system as the foundation of a secure network operating system. Technical Report TR-14-02-05. Department

- of Computer Sciences 4 (Distributed Systems and Operating Systems), University of Erlangen-Nuremberg. <https://www4.informatik.uni-erlangen.de/DE/Projects/JX/publications/jx-sec.pdf>
- [30] Google LLC. 2022. Project Sparrow: CantripOS. <https://github.com/AmibiML/sparrow-cantrip-full>.
- [31] Gary Guo. 2024. Unwinding: Unwinding library in Rust and for Rust. <https://github.com/nbdd0121/unwinding>.
- [32] Philipp Haller and Martin Odersky. 2010. Capabilities for uniqueness and borrowing. In *Proceedings of the 24th European Conference on Object-Oriented Programming (ECOOP)*. Springer, 354–378. doi:10.1007/978-3-642-14107-2_17
- [33] Norm Hardy. 1988. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36–38. doi:10.1145/54289.871709
- [34] Michael A Harrison, Walter L Ruzzo, and Jeffrey D Ullman. 1976. Protection in operating systems. *Commun. ACM* 19, 8 (1976), 461–471. doi:10.1145/360303.360333
- [35] Gernot Heiser. 2008. What is the difference between a microkernel? <https://microkerneldude.org/2008/03/13/q-what-is-the-difference-between-a-microkernel>.
- [36] Gernot Heiser. 2025. *The seL4 microkernel – An introduction*. White paper. seL4 Foundation. <https://sel4.systems/About/seL4-whitepaper.pdf>
- [37] Jorrit N Herder. 2005. *Towards a true microkernel operating system*. Master’s thesis. Vrije Universiteit Amsterdam. https://minix3.org/doc/herder_thesis.pdf
- [38] Jessica Hillert. 2019. *A comparison of the capability systems of Encore, Pony and Rust*. Bachelor’s thesis. Uppsala University. <https://uu.diva-portal.org/smash/get/diva2:1363822/FULLTEXT01.pdf>
- [39] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust verification by functional translation. *Proceedings of the ACM on Programming Languages* 6, ICFP (2022), 711–741. doi:10.1145/3547647
- [40] Galen Hunt, James Larus, Martin Abadi, Mark Aiken, Paul Barham, Manuel Fähndrich, Chris Hawblitzel, Orion Hodson, Steven Levi, Nick Murphy, Bjarne Steensgaard, David Tarditi, Ted Wobber, and Brian Zill. 2005. *An overview of the Singularity project*. Technical Report MSR-TR-2005-135. Microsoft Research. <https://www.microsoft.com/en-us/research/wp-content/uploads/2005/10/tr-2005-135.pdf>
- [41] Galen C Hunt and James R Larus. 2007. Singularity: Rethinking the software stack. *ACM SIGOPS Operating Systems Review* 41, 2 (2007), 37–49. doi:10.1145/1243418.1243424
- [42] Samuel Jero, Juliana Furgala, Runyu Pan, Phani Kishore Gadepalli, Alexandra Clifford, Bite Ye, Roger Khazan, Bryan C Ward, Gabriel Parmer, and Richard Skowyra. 2021. Practical principle of least privilege for secure embedded systems. In *Proceedings of the 27th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 1–13. doi:10.1109/RTAS52030.2021.00009
- [43] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages* 2, POPL (2018), 66:1–66:34. doi:10.1145/3158154
- [44] Kernkonzept GmbH. 2026. The L4Re Operating System Framework. <https://l4re.org>.
- [45] Gerwin Klein, June Andronick, Kevin Elphinstone, Toby Murray, Thomas Sewell, Rafal Kolanski, and Gernot Heiser. 2014. Comprehensive formal verification of an OS microkernel. *ACM Transactions on Computer Systems (TOCS)* 32, 1 (2014), 2:1–2:70. doi:10.1145/2560537
- [46] Kry10 Unlimited, Inc. 2024. Kry10 - Enabling innovation in high trust systems. <https://www.kry10.com/>.
- [47] Adam Lackorzynski and Alexander Warg. 2009. Taming subsystems: Capabilities as universal resource access control in L4. In *Proceedings of the 2nd Workshop on Isolation and Integration in Embedded Systems (IIES)*. ACM, 25–30. doi:10.1145/1519130.1519135
- [48] Butler W. Lampson. 1971. Protection. In *Proceedings of the 5th Princeton Symposium on Information Sciences and Systems*. 437–443. doi:10.1145/775265.775268 Reprinted in *Operating Systems Review*, 8(1):18–24, 1974.
- [49] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 286–315. doi:10.1145/3586037
- [50] Junghee Lee, Youngjae Kim, Galen M Shipman, Sarp Oral, and Jongman Kim. 2013. Preemptible I/O scheduling of garbage collection for solid state drives. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 32, 2 (2013), 247–260. doi:10.1109/TCAD.2012.227479
- [51] Nico Lehmann, Adam T Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid types for Rust. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 1533–1557. doi:10.1145/3591283
- [52] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B Giffin, Pat Pannuto, Prabal Dutta, and Philip Levis. 2017. Multiprogramming a 64kB computer safely and efficiently. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*. ACM, 234–251. doi:10.1145/3132747.3132786
- [53] Henry M Levy. 1984. *Capability-based computer systems*. Digital Press. doi:10.1016/C2013-0-01290-X
- [54] Zhaofeng Li, Tianjiao Huang, Vikram Narayanan, and Anton Burtsev. 2021. Understanding the overheads of hardware and language-based IPC mechanisms. In *Proceedings of the 11th Workshop on Programming Languages and Operating Systems (PLOS)*. ACM, 53–61. doi:10.1145/3477113.3487275
- [55] Jochen Liedtke. 1995. On micro-kernel construction. In *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 237–250. doi:10.1145/224056.224075
- [56] Jochen Liedtke. 1996. Toward real microkernels. *Commun. ACM* 39, 9 (1996), 70–77. doi:10.1145/234215.234473
- [57] Richard J Lipton and Lawrence Snyder. 1977. A linear time algorithm for deciding subject security. *J. ACM* 24, 3 (1977), 455–464. doi:10.1145/322017.322025
- [58] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: A semantic foundation for functional verification of Rust programs with unsafe code. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. ACM, 841–856. doi:10.1145/3519939.3523704
- [59] Adrian Mettler, David A Wagner, and Tyler Close. 2010. Joe-E: A security-oriented subset of Java. In *Proceedings of the 17th Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 357–374. <https://www.ndss-symposium.org/wp-content/uploads/2017/09/met.pdf>
- [60] Till Miemietz, Viktor Reusch, Matthias Hille, Max Kurze, Adam Lackorzynski, Michael Roitzsch, and Hermann Härtig. 2024. A perfect fit? - Towards containers on microkernels. In *Proceedings of the 10th International Workshop on Container Technologies and Container Clouds (WoC)*. ACM, 1–6. doi:10.1145/3702637.3702957
- [61] Till Miemietz, Viktor Reusch, Matthias Hille, Lars Wrenger, Jana Eisoldt, Jan Klötzke, Max Kurze, Adam Lackorzynski, Michael Roitzsch, and Hermann Härtig. 2025. MetEagle: Costs and benefits of implementing containers on microkernels. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 979–996. <https://www.usenix.org/system/files/osdi25-miemietz.pdf>
- [62] Mark Samuel Miller. 2006. *Robust composition: Towards a unified approach to access control and concurrency control*. Ph. D. Dissertation. Johns Hopkins University. <https://jscholarship.library.jhu.edu/serve/api/core/bitstreams/c2c3ea50-2613-47a7-b33c-9383c706ee63/content>

- tent
- [63] Mark S Miller, Chip Morningstar, and Bill Frantz. 2000. Capability-based financial instruments. In *Proceedings of the 4th International Conference on Financial Cryptography (FC)*. Springer, 349–378. doi:10.1007/3-540-45472-1_24
- [64] Mark S Miller, Ka-Ping Yee, and Jonathan Shapiro. 2003. *Capability myths demolished*. Technical Report SRL2003-02. Systems Research Laboratory, Johns Hopkins University. <http://zesty.ca/capmyths/usenix.pdf>
- [65] Toby Murray, Daniel Matichuk, Matthew Brassil, Peter Gammie, Timothy Bourke, Sean Seefried, Corey Lewis, Xin Gao, and Gerwin Klein. 2013. seL4: From general purpose to a proof of information flow enforcement. In *Proceedings of the 34th IEEE Symposium on Security and Privacy (SP)*. IEEE, 415–429. doi:10.1109/SP.2013.35
- [66] Vikram Narayanan, Tianjiao Huang, David Detweiler, Dan Appel, Zhaofeng Li, Gerd Zellweger, and Anton Burtsev. 2020. RedLeaf: Isolation and communication in a safe operating system. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 21–39. https://www.usenix.org/system/files/osdi20-narayanan_vikram.pdf
- [67] Yuke Peng, Hongliang Tian, Jinyi Xian, Shuai Zhou, Shoumeng Yan, and Yinqian Zhang. 2024. Framekernel: A safe and efficient kernel architecture via Rust-based intra-kernel privilege separation. In *Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys)*. ACM, 31–37. doi:10.1145/3678015.3680492
- [68] Yuke Peng, Hongliang Tian, Junyang Zhang, Ruihan Li, Chengjun Chen, Jianfeng Jiang, Jinyi Xian, Xiaolin Wang, Chenren Xu, Diyu Zhou, Yingwei Luo, Shoumeng Yan, and Yinqian Zhang. 2025. Assterinas: A Linux ABI-compatible, Rust-based framekernel OS with a small and sound TCB. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 307–323. <https://www.usenix.org/system/files/atc25-peng-yuke.pdf>
- [69] Ning Qu. 2024. seL4 in software-defined vehicles: Vision, roadmap, and impact at NIO. <https://sel4.systems/Summit/2024/slides/software-defined.pdf>. Keynote at the seL4 Summit 2024.
- [70] Jerome H Saltzer and Michael D Schroeder. 1975. The protection of information in computer systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939
- [71] Leon Schuermann, Brad Campbell, Branden Ghena, Philip Levis, Amit Levy, and Pat Pannuto. 2025. Tock: From research to securing 10 million computers. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. ACM, 36–49. doi:10.1145/3731569.3764828
- [72] seL4 Foundation. 2024. The seL4 Microkit. <https://docs.sel4.systems/projects/microkit/>.
- [73] seL4 Foundation. 2025. CAmkES VMM. <https://docs.sel4.systems/projects/camkes-vm/>.
- [74] seL4 Foundation. 2025. The sel4bench suite. <https://docs.sel4.systems/projects/sel4bench>.
- [75] Thomas Sewell, Simon Winwood, Peter Gammie, Toby Murray, June Andronick, and Gerwin Klein. 2011. seL4 enforces integrity. In *Proceedings of the 2nd International Conference on Interactive Theorem Proving (ITP)*. Springer, 325–340. doi:10.1007/978-3-642-22863-6_24
- [76] Jonathan S Shapiro, Jonathan M Smith, and David J Farber. 1999. EROS: A fast capability system. In *Proceedings of the 17th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 170–185. doi:10.1145/319151.319163
- [77] Jiacheng Shi, Jinyu Gu, Yubin Xia, and Haibo Chen. 2025. Serverless functions made confidential and efficient with split containers. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security)*. USENIX Association, 1091–1110. <https://www.usenix.org/system/files/usenixsecurity25-shi-jiacheng.pdf>
- [78] SQLite Consortium. 2025. SQLite. <https://www.sqlite.org/index.html>.
- [79] Marc Stiegler and Mark Miller. 2006. *How Emily tamed the Caml*. Technical Report. Hewlett-Packard Laboratories. <https://shiftleft.com/mirrors/www.hpl.hp.com/techreports/2006/HPL-2006-116.pdf>
- [80] David Swasey, Deepak Garg, and Derek Dreyer. 2017. Robust and compositional verification of object capability patterns. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 89:1–89:26. doi:10.1145/3133913
- [81] June Tate-Gans, Sam Leffler, and Kai Yick. 2023. CantripOS: An OS for Ambient ML applications. <https://sel4.systems/Foundation/Summit/2023/slides/day2-01-leffler.pdf>. Keynote at the seL4 Summit 2023.
- [82] The cppreference.com contributors. 2022. C++ named requirements: PODType (deprecated in C++20). https://en.cppreference.com/w/cpp/named_req/PODType.
- [83] The Redox Developers. 2018. RedoxFS: The Redox filesystem. <https://gitlab.redox-os.org/redox-os/redoxfs>.
- [84] The Redox Developers. 2025. Redox OS. <https://www.redox-os.org>.
- [85] Trustworthy Systems Research Group, UNSW Sydney. 2024. The Lions Operating System (LionsOS). <https://lionsos.org>.
- [86] Thorsten von Eicken, Chi-Chao Chang, Grzegorz Czajkowski, Chris Hawblitzel, Deyu Hu, and Dan Spoonhower. 1999. J-Kernel: A capability-based operating system for Java. In *Secure Internet Programming: Security Issues for Mobile and Distributed Objects*. Springer, 369–393. doi:10.1007/3-540-48749-2_17
- [87] Carsten Weinhold, Nils Asmussen, Diana Göhringer, and Michael Roitzsch. 2023. Towards modular trusted execution environments. In *Proceedings of the 6th Workshop on System Software for Trusted Execution (SysTEX)*. ACM, 10–16. doi:10.1145/3578359.3593037
- [88] Roland Wismüller, Damian Ludwig, and Felix Breitweiser. 2024. Extending the object-capability model with fine-grained type-based capabilities. *Journal of Object Technology* 23, 1 (2024), 1–36. doi:10.5381/JOT.2024.23.1.A1